

TP 1 – RPC Middleware : Remote Procedure Call (RPC), Architecture Client/Serveur

Objectifs :

- Initiation à la programmation distribuée (Architecture client/serveur).
- Introduction aux middlewares à appel de procédure distante.
- Manipulation de l'outil RPCGEN pour le développement des applications réparties en se basant sur SUN RPC.
- Comprendre l'utilité des middlewares pour la programmation des applications distribuées.

Outils :

- OS Linux Ubuntu version 12 et supérieur.
- rpcgen tool package for linux Ubuntu.

I. Les RPC de SUN : Open Network Computing (ONC)

L'Open Network Computing (ONC) Remote Procedure Call (RPC) est un système d'appel de procédure à distance. ONC a été initialement développé par Sun Microsystems dans les années 1980 dans le cadre de son projet de système de fichiers réseau (NFS), et est parfois appelé Sun RPC.

Exercices 1 : Récupérez l'archive **mmots.zip** qui contient tous les fichiers source en C, le fichier RPCL (IDL) mmots.x ainsi qu'un fichier Makefile.

- a. Commencez par regarder le Makefile afin de comprendre l'appel à rpcgen, et la création des exécutables du serveur et du client.
- b. Il ne reste plus qu'à lancer make, ce qui a pour effet de
 - faire générer par rpcgen (avec le fichier RPCL mmots.x)
 - le stub côté client mmots_clnt.c,
 - le skeleton côté serveur mmots_svc.c,
 - un fichier annexe mmots_xdr.c dont on ne se souciera pas pour ce premier exercice.
 - compiler le programme client mmots et le programme serveur mmotsd.

Test de l'application :

Maintenant on va faire quelques tests. Nous ferons tourner le serveur sur la machine locale. Le client pourra être lancé depuis n'importe quelle machine, y compris la machine locale.

Pour lancer le serveur :

./mmotsd

Et pour lancer le client :

./mmots **nom de la machine sur laquelle tourne le serveur**

- a. Vérifiez qu'un programme RPC a bien été enregistré à votre nom sur la machine.
- b. Dans le cas de plusieurs serveurs mmotsd lequel est choisi ?
- c. Vous pouvez ensuite entrer les commandes suivantes dans mmots :
 - I pour vider le dictionnaire
 - i pour insérer un mot
 - s pour supprimer un mot
 - c pour chercher si un mot existe
 - q pour quitter
- d. Utilisez à plusieurs le même serveur, et voyez l'effet de vos requêtes concurrentes.

Compréhension du code et ajout de fonctions

Il est important de bien comprendre la manière dont les différentes parties de l'application collaborent entre-elles. En particulier, il est important de faire la différence entre les fichiers générés par rpcgen et ceux fournis dans l'archive mmots.zip.

- Ajouter une fonction qui retourne le nombre de mots dans le dictionnaire.

Exercices 2 :

En utilisant l'outil RPCGEN, Ecrire une application distribuée qui permet d'incrémenter un nombre entier. Le client lance le calcul et l'opération d'incrémentation est réalisée sur le serveur, le résultat est renvoyé au client.

Exercices 3 :

En utilisant l'outil RPCGEN, Ecrire une application distribuée qui prend en entrée deux nombres entiers (a et b), elle rend en sortie : (a+b), (a/b), (a-b) et (a*b) au même temps.

PS : L'application contient une seule fonction.

II. Programmation RPC par sockets

Exercices 4 :

- Faire le même travail en utilisant les sockets (Java ou C).
- Expliquer la différence entre ces différentes manières du développement d'applications distribuées client/serveur.